Programming Languages and Paradigms: COMP 302

Instructor: Jacob Thomas Errington

Winter 2025

Welcome to COMP 302! This course is divided into three modules.

1. I will introduce you to a (presumably) unfamiliar programming language paradigm in a (presumably) unfamiliar programming language: *functional programming* (FP) in OCaml. In doing so, I hope to show you to appreciate the differences between FP and imperative programming, which you are already familiar with from your experience in Java, Python, or similar languages. Some aspects of FP will seem painfully similar to things you have seen before; others will be painfully different. But as one might say when working out: *no pain, no gain!*

2. We will survey language features that are common in many languages, such as exceptions, mutable variables, delayed evaluation, and sophisticated mechanisms for manipulating control flow. We will also learn how to use a powerful mathematical technique, proof by induction, to reason about the recursive programs that we write. For example, we will prove the soundness of certain kinds of transformations we can apply to programs and to outright prove certain correctness properties about the algorithms we develop.

3. I will demonstrate the fundamentals of *programming language design*. You will see how the syntax of a program is represented, manipulated, and interpreted in code. You will also see how we *formally* define the semantics of a programming language. That is, we will define a *type system* and some *evaluation rules* for our own little programming language. Of particular importance to us will be the representation of *variables* and operations concerning them such as *substitutions*, as this represents a major challenge in programming language theory and practice.

Logistics

Instructor. Jacob Errington, Faculty Lecturer, `jacob.errington@mcgill.ca`, McConnell Room 234A.

Classes. Adams Auditorium – Mondays and Wednesdays – 2:35 to 3:55 PM.
Please bring a computer as you will be doing problems in class!

Discord. We will use Discord for all communications outside of class, i.e. announcements and course discussion boards. Monitoring the Discord server's `#announcements` channel is mandatory.[1]
A link to join the server will be posted on MyCourses.

Office hours. See the Discord `#syllabus` channel.

Teaching assistants. See the Discord `#syllabus` channel.

[1] Although I will typically `@everyone`, I strongly recommend enabling notifications for all messages in that channel.

Evaluation

Homework ($10 \times 2\% = 20\%$)**.**

- There are be *ten assignments*, roughly weekly.
- These are *not* designed as assessments, but rather as *learning activities.*
- **To best learn the material in this course, I recommend working on the homework within 24h of the relevant concept being introduced, and generally completing it within one week.**
- Homework are delivered via the LearnOCaml platform[2], where you can both write code and run an auto-grader. This determines your score on the assignment. An assignment score of at least 80% earns the full marks for that assignment. Any score less than 80% does not earn any marks.
- **The final submission of homework is made via myCourses.**

Classtests ($3 \times 10\% = 30\%$)**.** Closed book; one US letter / A4 size crib sheet is allowed, handwritten only[3], one side only. During class hours. Each covers material from its respective module.
Exact dates and times TBA on Discord.

Final (50%)**.** Comprehensive. Consists of three evenly-weighted sections, one for each course module. Closed book. One *double-sided* cheat cheat, handwritten only[4], is allowed.
Date and time TBA by the university.

Evaluation flexibility

The following policies are designed to give you some flexibility in the evaluation in this course.

No homework due dates. The homework in this course are intended as weekly learning activites to help you solidify your understanding of the course material. I will not twist your arm with weekly deadlines to complete these, but I will strongly recommend that you do these on the day the concept is introduced.
The "hard" due date of all homework is the date of the final exam. No homework is accepted after that date, without exception.

Classtest-final weight shifting.

- If you cannot write a classtest *for any reason,* **you must notify me,**[5] and its weight fully shifts to the final exam.
- If the score on a final exam section exceeds the corresponding classtest's score, **half** of the classtest's weight shifts to *that section* of the final.
- If the score on a classtest exceeds the corresponding final exam section's score, **the full weight** of *that section* of the final exam shifts to the classtest.
- Therefore, if the scores achieved in all three classtests are to your satisfaction, **you do not have to write the final exam.**

[2] Link posted on Discord.

[3] This means physically written by your hand on paper. A printout of a page "handwritten" on an electronic device is not accepted. The exercise of carefully spatially synthesizing your knowledge is valuable. Students with SAA accommodation permitting them to type their exam or otherwise relating to a handwriting disability (e.g. dysgraphia) are exempt from the handwriting requirement.

[4] As above.

[5] See `#syllabus` for the mechanism how.

Teaching methods

Classes will not consist of only lecturing!

My belief, which is grounded in the research literature of computer science education, is that student-centered learning is not only more inclusive but also more effective. What that means is that in class, you can expect to solve problems, discuss in pairs or small groups, answer polls, and so on. You *will not* be scribbling notes while I blather on at breakneck pace – I still recommend taking notes – but rather have time to reflect on and properly construct your own understanding of the new ideas presented in class.

However, this means that you are expected to be an active participant in class. Of course, you *could* choose not to participate, (or even to not come to class – all lectures are recorded,) but I can assure you that not only will you not learn as well as the students who do attend classes, but you will also find classes incredibly boring if you choose to be passive.

Rough schedule

A more detailed schedule with a class-by-class breakdown is posted in `#schedule`.

First month. *FP fundamentals:* expressions, values, types, pattern matching, recursion, lists, higher-order functions, continuations. Reasoning about programs: tracing, induction.

Second month. *Language features:* exceptions, modules, mutable state, lazy evaluation. Object-oriented programming.

Third month. *Language design:* free/bound variables, scoping, substitution, evaluation, typechecking, polymorphism, subtyping.

Homework

Homework assignments will roughly cover the material from the week's classes. There are a lot of assignments, and as such, they are designed to take half as much time to do than as if there were half as many.

Although you can do your homework on the LearnOCaml platform, and click the "grade" button on the platform to see what your grade *would* be, **you must submit the version of the assignment that you want to have graded on MyCourses.** Instructions on how to do so will be given later in the semester as the "hard" due date of the homework approaches.

Safe space

I am committed to nurturing a space where students, teaching assistants, lecturers, and professors can all engage in the exchange of ideas and dialogue, without fear of being made to feel unwelcome or unsafe on account of biological sex, sexual orientation, gender identity or expression, race/ethnicity, religion, linguistic and cultural background, age, physical or mental ability, or any other aspect integral to one's personhood. I therefore recognize our responsibility, both individual and collective, to strive to establish and maintain an environment wherein all interactions are based on empathy and mutual respect for the person, acknowledging differences of perspectives, free from judgment, censure, and stigma.

If you should feel unsafe or unwelcome in class, in office hours, or in an exam, as a result of the actions or behaviour of your fellow students, of a TA, or of me, then **please reach out to me** or to the school ombudsperson. **I take this seriously.**

Copyright

Instructor-generated course materials (e.g., handouts, notes, summaries, homework, exam questions, etc.) are protected by law and may not be copied or distributed in any form or in any medium without explicit permission of the instructor. Note that infringements of copyright can be subject to followup by the University under the Code of Student Conduct and Disciplinary Procedures.

Language rights

In accord with McGill University's Charter of Students' Rights, students in this course have the right to submit in English or in French any written work that is to be graded.

Academic integrity

McGill University values academic integrity. Therefore all students must understand the meaning and consequences of cheating, plagiarism and other academic offenses under the Code of Student Conduct and Disciplinary Procedures (see `http://www.mcgill.ca/integrity/` for more information).

Most importantly, work submitted for this course must represent your own efforts. Copying assignments or tests from any source, completely or partially, or allowing others to copy your work, will not be tolerated.

Regarding generative AI tools, e.g. ChatGPT, you are permitted but discoraged to use these for homework. The homework are the main practice problems in this course, and are the primary mechanism by which you will solidify an understanding to demonstrate in invigilated tests, from which is derived the bulk of your grade in this course. If you do not do the homework yourself, then it is unlikely that you will find satisfaction in your performance on the tests.